

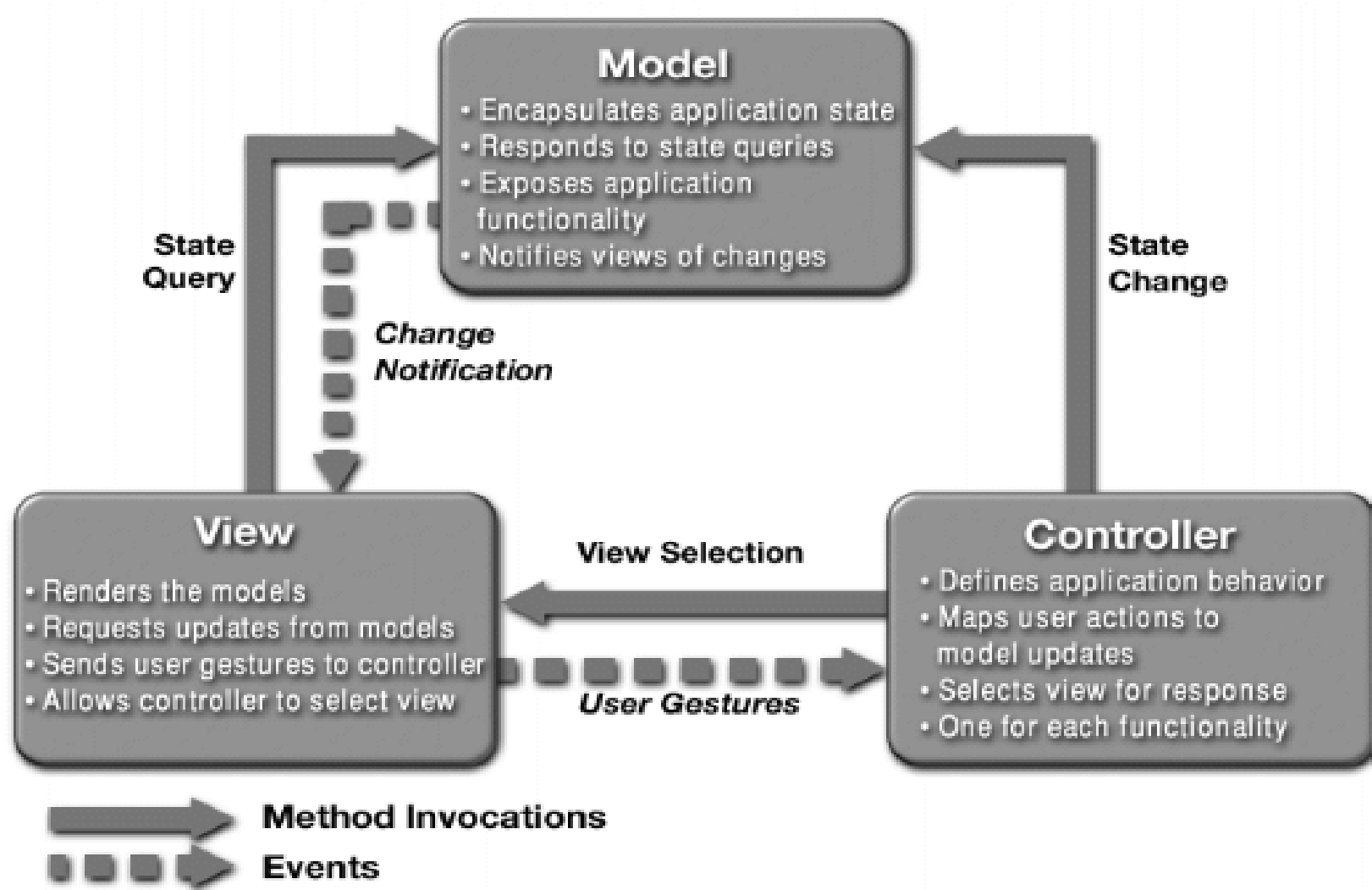
Component-and-connect patterns:

- Broker
- **Model-view Controller**
- Pipe and Filter
- Client-Server
- Peer-to-Peer
- Service-Oriented-Architecture
- Publish-Subscribe
- Shared-Data

Model-View-Controller Pattern

- **Context:** User interface software is typically the most frequently modified portion of an interactive application. Users often wish to look at data from different perspectives, such as a bar graph or a pie chart. These representations should both reflect the current state of the data.
- **Problem:** How can user interface functionality be kept separate from application functionality and yet still be responsive to user input, or to changes in the underlying application's data? And how can multiple views of the user interface be created, maintained, and coordinated when the underlying application data changes?
- **Solution:** The model-view-controller (MVC) pattern separates application functionality into three kinds of components:
 - A model, which contains the application's data
 - A view, which displays some portion of the underlying data and interacts with the user
 - A controller, which mediates between the model and the view and manages the notifications of state changes

MVC Example



MVC Solution - 1

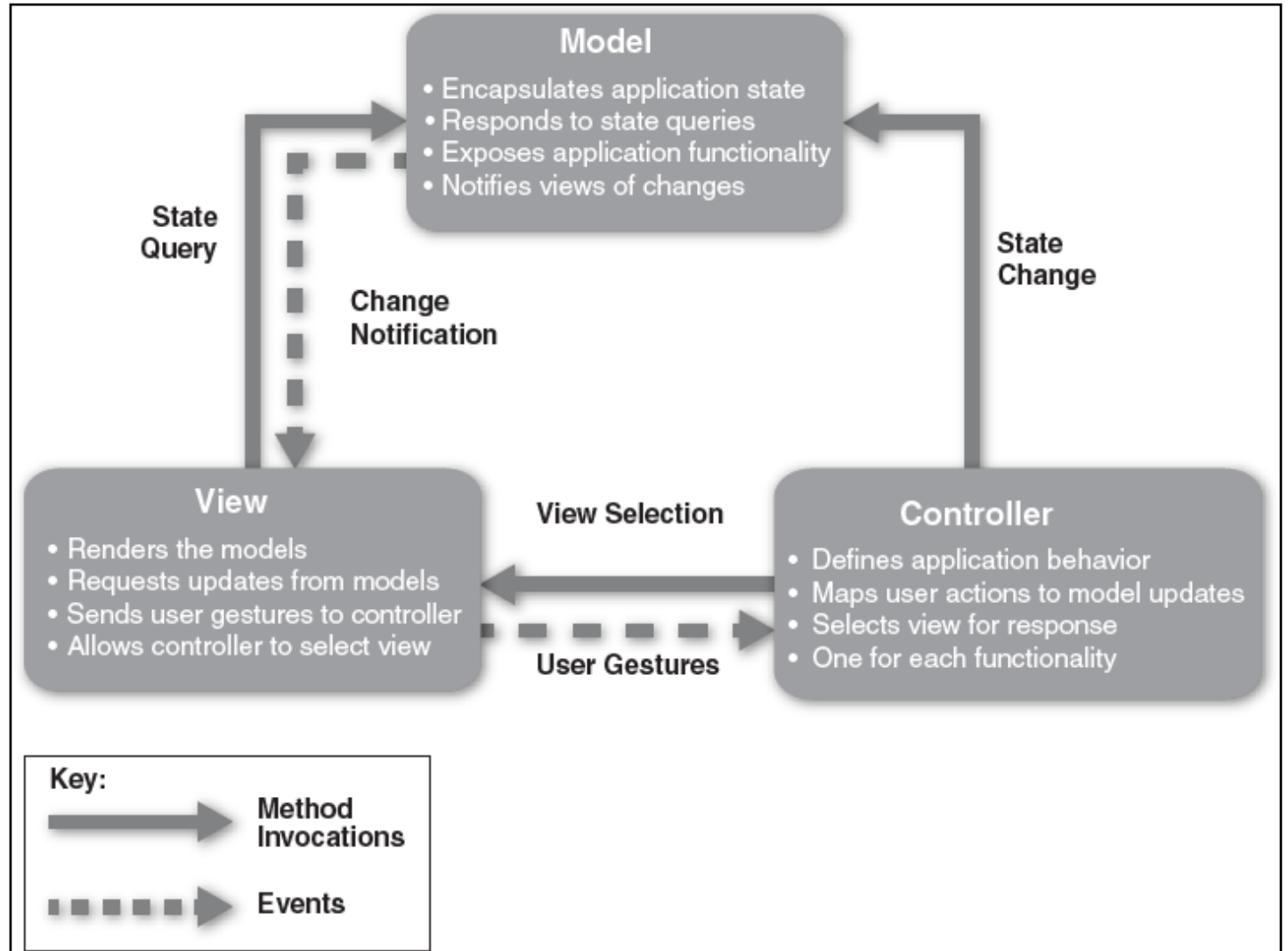
- Overview: The MVC pattern breaks system functionality into three components: a model, a view, and a controller that mediates between the model and the view.
- **Elements:**
 - The *model* is a representation of the application data or state, and it contains application logic.
 - The *view* is a user interface component that either produces a representation of the model for the user or allows for some form of user input, or both.
 - The *controller* manages the interaction between the model and the view, translating user actions into changes to the model or changes to the view.

MVC Solution - 2

- **Relations:** The *notifies* relation connects instances of model, view, and controller, notifying elements of relevant state changes.
- **Constraints:**
 - There must be at least one instance each of model, view, and controller.
 - The model component should not interact directly with the controller.
- **Weaknesses:**
 - The complexity may not be worth it for simple user interfaces.
 - The model, view, and controller abstractions may not be good fits for some user interface toolkits.
 - The view and the controller split apart input and output, but these functions are often combined into individual widgets. This may result in a conceptual mismatch between the architecture and the user interface toolkit.

Finer Points for MVC

- The MVC components are connected to each other via some flavor of notification, such as events or callbacks.
 - These notifications contain state updates.
- A change in the model needs to be communicated to the views so that they may be updated.
- An external event, such as a user input, needs to be communicated to the controller, which may in turn update the view and/or the model.
- Because the MVC components are loosely coupled, it is easy to develop and test them in parallel, and changes to one have minimal impact on the others.



Finer Points for MVC

- There may be many views and many controllers associated with a model.
- **Many views for a model:** For example, a set of business data may be represented as columns of numbers in a spreadsheet, as a scatter plot, or as a pie chart.
 - Each of these is a separate view, and this view can be dynamically updated as the model changes
 - (for example, showing live transactions in a transaction processing system).
- **Many controllers for a model** A model may be updated by different controllers;
 - for example, a map could be zoomed and panned via mouse movements, trackball movements, keyboard clicks, or voice commands
 - each of these different forms of input needs to be managed by a controller.

Usage Examples of MVC

- The MVC pattern is widely used in user interface libraries such as:
 - Java's Swing classes
 - Microsoft's ASP.NET framework